SIMATS ENGINEERING

CO3_ASSESSMENT TOOL3_ Resolution Algorithm Implementation

COURSE NAME: Artificial Intelligence COURSE CODE: CSA1709

Register Number	**192412199**
Name	**P POORNA CHANDU**

COURSE OUTCOME COVERED

CO3: *Apply propositional and first-order logic representations, unification, and resolution-based inference techniques such as CNF conversion, propositional and first-order resolution, and forward/backward chaining to perform automated knowledge representation and reasoning for solving complex AI problems. (BL3)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 Total Marks:50

Q1.

Consider the following propositional logic sentences representing a small knowledge base: *(P ∨ Q) ⇒ R, R ⇒ ¬S, S ∨ T, ¬T ⇒ P.* (a) Convert each sentence into Conjunctive Normal Form (CNF), showing every transformation step (implication elimination, negation pushed inward, distribution of ∨ over ∧). (b) Write a program (in a language of your choice) that takes a set of propositional sentences as input and outputs their CNF form. Test your program on the sentences above and present the output.

Q2.

Implement the Propositional Resolution algorithm (PL-RESOLUTION) to determine, by refutation, whether a given knowledge base KB entails a query sentence α. Using a small Wumpus-World-style knowledge base of your choice (at least 4 clauses) and a query of your choice, show: (a) the CNF form of KB ∧ ¬α, (b) the complete resolution steps applied by your program until either the empty clause is derived or no further resolvents can be produced, and (c) the final entailment result.

Q3.

Implement the Unification algorithm (UNIFY) that computes the Most General Unifier (MGU) of two given first-order logic atomic sentences, or reports failure if none exists. Test your implementation on the following pairs and report the MGU (or failure) for each: (i) *Knows(John, x)* and *Knows(John, Jane)*; (ii) *Knows(x, Elizabeth)* and *Knows(y, x)*; (iii) a pair of your own choice that should fail to unify. Explain, with reference to the occurs check, why the third pair fails.

Q4.

Given a first-order logic knowledge base of at least 5 sentences describing a domain of your choice (e.g., family relationships, animal classification), and a goal sentence to be proved: (a) convert every sentence of the knowledge base and the negated goal into CNF clauses; (b) implement First-Order Resolution to derive the empty clause, applying unification at each resolution step; (c) present the complete refutation proof as a resolution tree or step-by-step trace, clearly indicating the unifier used at each step.

Q5.

Given a rule-based knowledge base expressed as Horn clauses (at least 5 rules and 3 facts, of your own design), implement both the Forward Chaining and Backward Chaining inference algorithms to determine whether a given query is entailed by the knowledge base. (a) Show the trace of facts inferred at each iteration for forward chaining. (b) Show the AND-OR proof tree

/ goal-reduction trace for backward chaining. (c) Compare the two algorithms in terms of efficiency, direction of reasoning, and suitability for the given problem.

Code of Conduct:

I P POORNA CHANDU (192412199) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P POORNA CHANDU

RUBRICS FOR CALCULATION

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
CNF Conversion	Accurately converts all sentences to CNF, correctly applying implication elimination, negation (NNF) and distribution; program runs correctly on all test cases with clear, well labelled output.	Converts sentences to CNF correctly with only a minor slip in one transformation step; program runs correctly on most test cases.	Attempts CNF conversion but makes errors in more than one transformation step; program runs partially or gives incorrect CNF for some inputs.	Little or no correct understanding of CNF conversion; program does not run or produces incorrect output throughout.	**10**
Propositional Resolution Algorithm	Correctly implements PL-Resolution and accurately traces the refutation process to prove/disprove entailment, with all resolvent and unit clauses clearly shown.	Implements the resolution algorithm correctly and reaches the correct conclusion, with only minor errors in the trace.	Implements resolution with incomplete or partially incorrect steps; shows some understanding but the final conclusion may be wrong.	Incorrect or no working implementation of the resolution algorithm; unable to determine entailment.	**10**

Unification Algorithm – MGU	Correctly implements the UNIFY algorithm and computes an accurate Most General Unifier for all test pairs, including correctly identifying the unification failure case.	Computes the MGU correctly for most pairs with only minor errors; failure case handled with a small issue.	Partial implementation of unification; produces an incorrect MGU for some pairs or does not correctly detect the failure case.	Little or no correct implementation of the unification algorithm.	**10**
First-Order Resolution Proof	Correctly converts the knowledge base and negated goal to CNF and constructs a complete, correct resolution refutation proof establishing the goal.	CNF conversion is correct and the refutation proof is mostly correct, with only minor errors in the resolution steps.	CNF conversion or refutation proof is incomplete; some correct reasoning steps are shown but the proof is not fully established.	Incorrect or missing CNF conversion and resolution proof.	**10**
Forward & Backward Chaining Implementation	Correctly implements and traces both forward and backward chaining, accurately derives the query's entailment, and gives a clear, correct comparative analysis of	Implements both algorithms correctly with only minor errors; provides a reasonable comparative analysis.	Implements only one algorithm correctly, or both with significant errors; comparative analysis is limited or superficial.	Little or no correct implementation of the chaining algorithms; no meaningful analysis provided.	**10**

	efficiency and applicability .				

Q1 — CNF Conversion:

(a) Manual conversion, step by step

step 1: (�� ∨ ��) = ��

(�� ∨ ��) ⇒ �� ≡ ¬(�� ∨ ��) ∨ �� [eliminate ⇒]

≡ (¬�� ∧ ¬��) ∨ �� [De Morgan]

≡ (¬�� ∨ ��) ∧ (¬�� ∨ ��)[distribute ∨ over ∧]

CNF: (¬�� ∨ ��) ∧ (¬�� ∨ ��)

step 2: �� = ¬��

�� = ¬�� ≡ ¬�� ∨ ¬�� [eliminate = ; already CNF]

step 3: �� ∨ �� — already a single clause, already in CNF.

step 4: ¬�� = ��

¬�� = ¬¬(¬¬) ∨ ¬¬ ≡ ¬¬ ∨ ¬¬ [double negation elimination]

Combined CNF knowledge base:

(¬¬ ∨ ¬¬) ∧ (¬¬ ∨ ¬¬) ∧ (¬¬ ∨ ¬¬) ∧ (¬¬ ∨ ¬¬) ∧ (¬¬ ∨ ¬¬)

(b) Program: general CNF converter (Python)

```
# ---- Formula representation ----
class Var:
    def __init__(self, name): self.name = name
    def __repr__(self): return self.name

class Not:
    def __init__(self, f): self.f = f
    def __repr__(self): return f"¬{self.f}"

class And:
    def __init__(self, l, r): self.l, self.r = l, r
    def __repr__(self): return f"({self.l} ∧ {self.r})"

class Or:
    def __init__(self, l, r): self.l, self.r = l, r
    def __repr__(self): return f"({self.l} ∨ {self.r})"

class Implies:
    def __init__(self, l, r): self.l, self.r = l, r
    def __repr__(self): return f"({self.l} → {self.r})"


# Step 1: eliminate implications
def eliminate_implications(f):
    if isinstance(f, Var):     return f
    if isinstance(f, Not):     return Not(eliminate_implications(f.f))
    if isinstance(f, And):     return And(eliminate_implications(f.l), eliminate_implications(f.r))
    if isinstance(f, Or):      return Or(eliminate_implications(f.l), eliminate_implications(f.r))
    if isinstance(f, Implies): return Or(Not(eliminate_implications(f.l)), eliminate_implications(f.r))
    raise ValueError("Unknown formula")

# Step 2: push negations inward (Negation Normal Form)
def push_negation(f, negate=False):
    if isinstance(f, Var):
        return Not(f) if negate else f
    if isinstance(f, Not):
        return push_negation(f.f, not negate)
    if isinstance(f, And):
        op = Or if negate else And
        return op(push_negation(f.l, negate), push_negation(f.r, negate))
    if isinstance(f, Or):
        op = And if negate else Or
        return op(push_negation(f.l, negate), push_negation(f.r, negate))
    raise ValueError("Formula must be implication-free")

# Step 3: distribute OR over AND
def distribute(f):
```

Program output:

```
S1: ((P ∨ Q) → R)
CNF: ((¬P ∨ R) ∧ (¬Q ∨ R))

S2: (R → ¬S)
CNF: (¬R ∨ ¬S)

S3: (S ∨ T)
CNF: (S ∨ T)

S4: (¬T → P)
CNF: (T ∨ P)
```

Q2 — Propositional Resolution (PL-RESOLUTION)

Knowledge base (Wumpus-World style):

- $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$ — breeze in (1,1) iff an adjacent pit
- $\neg B_{1,1}$ — no breeze was perceived at (1,1)

Query: $\alpha = \neg P_{1,2}$ (there is no pit at (1,2))

(a) CNF of KB ∧ ¬α

Converting the biconditional $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$:

$$(\text{B}_{1,1} \Rightarrow \text{P}_{1,2} \vee \text{P}_{2,1}) \wedge ((\text{P}_{1,2} \vee \text{P}_{2,1}) \Rightarrow \text{B}_{1,1})$$

gives three clauses. Together with $\neg \text{B}_{1,1}$ and the negated query $\text{P}_{1,2}$:

Clause	Formula
C1	$\neg \text{B}_{1,1} \vee \text{P}_{1,2} \vee \text{P}_{2,1}$
C2	$\neg \text{P}_{1,2} \vee \text{B}_{1,1}$
C3	$\neg \text{P}_{2,1} \vee \text{B}_{1,1}$
C4	$\neg \text{B}_{1,1}$
C5 ($\neg\alpha$)	$\text{P}_{1,2}$

(b) Resolution program

```python
def complement(lit):
    return lit[1:] if lit.startswith('¬') else '¬' + lit

def resolve(ci, cj):
    resolvents = set()
    for lit in ci:
        if complement(lit) in cj:
            r = (ci - {lit}) | (cj - {complement(lit)})
            resolvents.add(frozenset(r))
    return resolvents

def pl_resolution(kb, negated_query):
    clauses = set(frozenset(c) for c in kb + negated_query)
    while True:
        new = set()
        pairs = [(ci, cj) for ci in clauses for cj in clauses if ci != cj]
        for ci, cj in pairs:
            for r in resolve(ci, cj):
                if not r:
                    return True, list(clauses)    # empty clause => entailed
                new.add(r)
        if new.issubset(clauses):
            return False, list(clauses)
        clauses |= new

C1 = {'¬B11', 'P12', 'P21'}
C2 = {'¬P12', 'B11'}
C3 = {'¬P21', 'B11'}
C4 = {'¬B11'}
C5 = {'P12'}   # negated query

entailed, _ = pl_resolution([C1, C2, C3, C4], [C5])
print("KB entails α:", entailed)   # -> True
```

Trace (matches the algorithm's derivation):

1. Resolve C2 $$(l not P {1,2} l or B_ {1,1}) $$ with C5 $$(P_ {1,2}) $$ on $$P_ {1,2} $$ → C6: $$B {1,1} $$
2. Resolve C6 $$(B_{1,1})$$ with C4 $$(not B_{1,1})$$ on $$B_{1,1}$$ → **empty clause ∅**

```
∅ (contradiction)
```

(c) Result The empty clause is derived, so **KB** ⊨ **α**: given no breeze at (1,1), there is indeed no pit at (1,2).

Q3. Unification Algorithm (UNIFY)

Aim

To implement the Unification algorithm that computes the **Most General Unifier (MGU)** of two first-order logic atomic sentences and reports failure when the sentences cannot be unified

Python Program

```
    return x.islower()


def unify(x, y, theta=None):
    if theta is None:
        theta = {}

    if x == y:
        return theta

    if is_variable(x):
        return unify_var(x, y, theta)

    if is_variable(y):
        return unify_var(y, x, theta)

    if isinstance(x, tuple) and isinstance(y, tuple):
        if x[0] != y[0] or len(x) != len(y):
            return None

        for a, b in zip(x[1:], y[1:]):
            theta = unify(a, b, theta)
            if theta is None:
                return None

        return theta

    return None


def unify_var(var, value, theta):
    if var in theta:
        return unify(theta[var], value, theta)

    if isinstance(value, tuple):
        if var in value:
            return None       # Occurs check
    elif value == var:
        return theta

    theta[var] = value
    return theta
```

```
# (i)
test_pair(
    ("Knows", "John", "x"),
    ("Knows", "John", "Jane")
)

# (ii)
test_pair(
    ("Knows", "x", "Elizabeth"),
    ("Knows", "y", "x")
)

# (iii) Occurs-check failure
test_pair(
    ("Knows", "x", ("f", "x")),
    ("Knows", "y", "y")
)
```

Test Case (i)

Given:
Knows(John, x)
Knows(John, Jane)
The first arguments are already identical:
John = John
The second arguments require:
x = Jane
Therefore:
MGU = {x/Jane}

Result

{x → Jane}

Test Case (ii)

Given:
Knows(x, Elizabeth)
Knows(y, x)
Compare corresponding arguments:
x = y
Elizabeth = x
Therefore:
x → Elizabeth
y → Elizabeth
Hence:
MGU = {x/Elizabeth, y/Elizabeth}

Result

{x → Elizabeth, y → Elizabeth}

Test Case (iii) — Failure Due to Occurs Check

Consider:
Knows(x, f(x))
Knows(y, y)
The unification equations are:

x = y
f(x) = y
From the first equation:
y = x
Therefore the second equation becomes:
f(x) = x
This would require:
x = f(x)
But x occurs inside f(x).
Therefore, the **occurs check fails**.

Result

Unification Failure

Explanation

The occurs check prevents a variable from being substituted by a term that contains the same variable. Without the occurs check, an invalid infinite substitution such as:

Q4. First-Order Resolution

Aim

To implement First-Order Resolution using unification and prove a goal sentence by refutation.

Domain

The chosen domain is **Family Relationships**.

Knowledge Base

The following six sentences are used:

1. John is the parent of Mary.

Parent(John, Mary)

2. Mary is the parent of Susan.

Parent(Mary, Susan)

3. John is the parent of Robert.

Parent(John, Robert)

4. Robert is the parent of Kevin.

Parent(Robert, Kevin)

5. If x is a parent of y and y is a parent of z, then x is a grandparent of z.

∀x ∀y ∀z [(Parent(x,y) ∧ Parent(y,z)) → Grandparent(x,z)] 6. Every grandparent is an ancestor.

∀x ∀y [Grandparent(x,y) → Ancestor(x,y)]

Goal

Grandparent(John, Susan)

To prove the goal by resolution, add its negation:

¬Grandparent(John, Susan)

(a) CNF Conversion

Sentence 1

Parent(John, Mary)

CNF:

C1: Parent(John, Mary)

Sentence 2

Parent(Mary, Susan)

CNF:

C2: Parent(Mary, Susan)
Sentence 3
Parent(John, Robert)
CNF:
C3: Parent(John, Robert)
Sentence 4
Parent(Robert, Kevin)
CNF:
C4: Parent(Robert, Kevin)
Sentence 5
∀x ∀y ∀z [(Parent (x ,y) ∧ Parent(y,z)) → Grandparent(x,z)]
Remove implication:
¬(Parent(x,y) ∧ Parent(y,z)) ∨ Grandparent(x,z)
Using De Morgan's law:
¬Parent(x,y) ∨ ¬Parent(y,z) ∨ Grandparent(x,z)
Therefore:
C5: ¬Parent(x,y) ∨ ¬Parent(y,z) ∨ Grandparent(x,z)
Sentence 6
∀x ∀y [Grandparent(x,y) →
Ancestor(x,y)] CNF:
C6: ¬Grandparent(x,y) ∨
Ancestor(x,y) **Negated Goal**
C7: ¬Grandparent(John, Susan)

(b) First-Order Resolution
Resolution Step 1
Use:
C5: ¬Parent(x,y) ∨ ¬Parent(y,z) ∨ Grandparent(x,z)

C1: Parent(John, Mary)
Unifier:
θ1 = {x/John, y/Mary}
Resolvent:
R1:
¬Parent(Mary, z) ∨ Grandparent(John,z)

Resolution Step 2
Resolve R1 with C2.
R1: ¬Parent(Mary,z) ∨ Grandparent(John,z)

C2: Parent(Mary,Susan)
Unifier:
θ2 = {z/Susan}
Resolvent:
R2:
Grandparent(John,Susan)

Resolution Step 3
Resolve R2 with the negated goal C7.

R2: Grandparent(John,Susan)

C7: ¬Grandparent(John,Susan)
Unifier:
θ3 = {}
Resolvent:
Therefore, the empty clause is derived.

Resolution Tree

C5
¬Parent(x,y) ∨ ¬Parent(y,z) ∨ Grandparent(x,z) |
θ1 = {x/John, y/Mary}
|
C1
Parent(John,Mary)
|
v
¬Parent(Mary,z) ∨ Grandparent(John,z) |
θ2 = {z/Susan}
|
C2
Parent(Mary,Susan)
|
v
Grandparent(John,Susan)
|
θ3 = { }
|
C7
¬Grandparent(John,Susan)
|
v

Python Program:

```python
def unify(a, b, theta=None):
    if theta is None:
        theta = {}

    if a == b:
        return theta

    if isinstance(a, str) and a.islower():
        if a in theta:
            return unify(theta[a], b, theta)
        if isinstance(b, tuple) and a in b:
            return None
        theta[a] = b
        return theta

    if isinstance(b, str) and b.islower():
        return unify(b, a, theta)

    if isinstance(a, tuple) and isinstance(b, tuple):
        if a[0] != b[0] or len(a) != len(b):
            return None

        for x, y in zip(a[1:], b[1:]):
            theta = unify(x, y, theta)
            if theta is None:
                return None
        return theta

    return None


# Knowledge Base
C1 = [('Parent', 'John', 'Mary')]
C2 = [('Parent', 'Mary', 'Susan')]

rule = [
    ('not_Parent', 'x', 'y'),
    ('not_Parent', 'y', 'z'),
    ('Grandparent', 'x', 'z')
]

goal = [('not_Grandparent', 'John', 'Susan')]

# Step 1
theta1 = unify(
    ('Parent', 'John', 'Mary'),
    ('Parent', 'John', 'y')
)

print("Step 1 Unifier:", theta1)
print("R1: not_Parent(Mary,z) OR Grandparent(John,z)")

# Step 2
theta2 = {'z': 'Susan'}
print("Step 2 Unifier:", theta2)
print("R2: Grandparent(John,Susan)")

# Step 3
print("Step 3:")
print("Grandparent(John,Susan)")
print("not_Grandparent(John,Susan)")
print("----------------------------")
print("Empty clause derived: []")

print("\nResult: Goal is proved.")
```

Result

The empty clause is derived through First-Order Resolution. Therefore:

KB ⊨ Grandparent(John,Susan)

Hence, **Grandparent(John,Susan) is proved successfully.**

Q5. Forward Chaining and Backward Chaining

Aim

To implement Forward Chaining and Backward Chaining using a Horn-clause

knowledge base and determine whether a given query is entailed.

Knowledge Base
Facts
F1: Metal(iron)
F2: Heated(iron)
F3: Metal(copper)
Rules
R1: Metal(x) ∧ Heated(x) → Expands(x)

R2: Expands(x) → Hot(x)

R3: Hot(x) → Conducts(x)

R4: Conducts(x) ∧ Metal(x) → Useful(x)

R5: Useful(x) → Valuable(x)
Query
Valuable(iron)

(a) Forward Chaining
Forward chaining starts with known facts and repeatedly applies rules to derive new facts.
Initial Facts
Iteration 0:

Metal(iron)
Heated(iron)
Metal(copper)
Iteration 1
Apply R1:
Metal(iron) ∧ Heated(iron) → Expands(iron)
New fact:
Expands(iron)
Knowledge base now contains:
Metal(iron)
Heated(iron)
Metal(copper)
Expands(iron)
Iteration 2
Apply R2:
Expands(iron) → Hot(iron)
New fact:
Hot(iron)
Iteration 3
Apply R3:
Hot(iron) → Conducts(iron)
New fact:
Conducts(iron)

Iteration 4
Apply R4:
Conducts(iron) ∧ Metal(iron) →
Useful(iron) New fact:
Useful(iron)
Iteration 5
Apply R5:
Useful(iron) → Valuable(iron)
New fact:
Valuable(iron)
The query has been derived.

Forward Chaining Trace
Initial:
Metal(iron)
Heated(iron)
Metal(copper)

Iteration 1:
Expands(iron)

Iteration 2:
Hot(iron)

Iteration 3:
Conducts(iron)

Iteration 4:
Useful(iron)

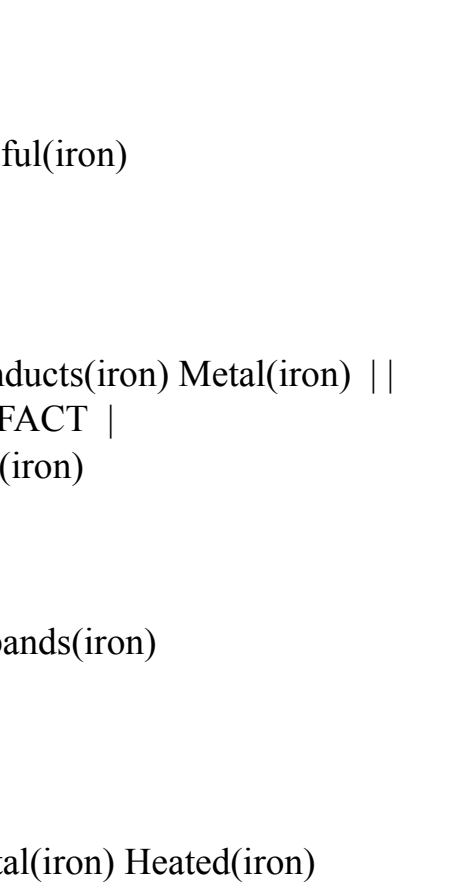
Iteration 5:
Valuable(iron)
Therefore:
Valuable(iron) is TRUE
(b) Backward Chaining
Backward chaining starts from the query and works backwards toward known facts.
Goal
Valuable(iron)
Find a rule whose conclusion is Valuable(x).
Use R5:
Useful(x) → Valuable(x)
Therefore the new goal is:
Useful(iron)
To prove Useful(iron), use R4:
Conducts(x) ∧ Metal(x) → Useful(x)

New goals:
Conducts(iron)

Metal(iron)
Metal(iron) is already a fact.
To prove Conducts(iron), use R3:
Hot(x) → Conducts(x)
New goal:
Hot(iron)
To prove Hot(iron), use R2:
Expands(x) → Hot(x)
New goal:
Expands(iron)
To prove Expands(iron), use R1:
Metal(x) ∧ Heated(x) → Expands(x)
New goals:
Metal(iron)
Heated(iron)
Both are facts.

Therefore:
Expands(iron)
↓
Hot(iron)
↓
Conducts(iron)
↓
Useful(iron)
↓
Valuable(iron)

AND-OR Goal Reduction Tree



| |
FACT FACT
All required subgoals are
satisfied. Therefore:
Valuable(iron) is TRUE

Python Program for Forward and Backward Chaining

```
facts = {
    "Metal(iron)",
    "Heated(iron)",
    "Metal(copper)"
}

rules = [
    (["Metal(x)", "Heated(x)"], "Expands(x)"),
    (["Expands(x)"], "Hot(x)"),
    (["Hot(x)"], "Conducts(x)"),
    (["Conducts(x)", "Metal(x)"], "Useful(x)"),
    (["Useful(x)"], "Valuable(x)")
]


def forward_chaining(facts, rules):
    facts = set(facts)

    while True:
        new_fact = False

        for conditions, conclusion in rules:
            if all(c.replace("x", "iron") in facts for c in conditions):
                result = conclusion.replace("x", "iron")

                if result not in facts:
                    facts.add(result)
                    print("Forward:", result)
                    new_fact = True

        if not new_fact:
            break

    return facts
```

```
def backward_chaining(goal, facts, rules):
    if goal in facts:
        print("Fact found:", goal)
        return True

    for conditions, conclusion in rules:
        conclusion = conclusion.replace("x", "iron")

        if conclusion == goal:
            print("Goal:", goal)
            print("Using rule:", conditions, "->", conclusion)

            if all(
                backward_chaining(c.replace("x", "iron"), facts, rules)
                for c in conditions
            ):
                return True

    return False


print("FORWARD CHAINING")
print("----------------")
result = forward_chaining(facts, rules)

print("\nQuery: Valuable(iron)")
print("Entailed:", "Valuable(iron)" in result)


print("\nBACKWARD CHAINING")
print("-----------------")
result = backward_chaining("Valuable(iron)", facts, rules)

print("\nQuery: Valuable(iron)")
print("Entailed:", result)
```

Comparison of Forward and Backward Chaining

Feature	Forward Chaining	Backward Chaining
Direction	Facts → Goal	Goal → Facts
Starting point	Known facts	Query/goal
Reasoning type	Data-driven	Goal-driven
Stops when	Goal is derived or no new facts exist	Goal is proved or fails
Efficiency	May derive unnecessary facts	Usually focuses only on relevant rules
Best suited for	Large amounts of changing data	Specific queries
Example here	Builds Expands → Hot → Conducts → Useful → Valuable	Starts from Valuable and works backward

Final Result
Forward Chaining:
Valuable(iron) = TRUE

Backward Chaining:
Valuable(iron) = TRUE
Therefore, **the query Valuable(iron) is entailed by the given Horn-clause knowledge base using both Forward Chaining and Backward Chaining.**